

M15 / LEARNER BOOK

Azure and Microsoft Fabric

Standalone Zero-to-Job-Ready Data Engineer 2026

16 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE15-01 | Cloud fundamentals for data engineers 4

DE15-02 | Identity, IAM and least privilege 8

DE15-03 | Azure Storage concepts and OneLake 11

DE15-04 | Fabric workspace and item model 14

DE15-05 | OneLake paths, files, tables and shortcuts 17

DE15-06 | Fabric Lakehouse and Delta tables 20

DE15-07 | Fabric notebooks 23

DE15-08 | Spark in Fabric 26

DE15-09 | Fabric Data Pipelines 29

DE15-10 | SQL analytics endpoint and warehouse choice 32

DE15-11 | Medallion architecture in Fabric 36

DE15-12 | Incremental loads and MERGE in cloud 39

DE15-13 | Permissions, connections and secrets 43

DE15-14 | Monitoring, reliability and cost awareness 46

DE15-15 | Git, deployment and environment separation 49

DE15-16 | End-to-end Fabric data engineering case 52

START HERE

How to use this module

Open the matching practice folder from the course root: 14_M15/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Beginner-first cloud engineering: Azure/Fabric boundaries, IAM, OneLake, workspaces/items, shortcuts, Lakehouse/Delta, notebooks, Spark Runtime 2.0, Data Factory pipelines, SQL analytics endpoint versus Warehouse, medallion, incremental MERGE, secrets, monitoring/cost, Git/deployment, and one end-to-end Fabric case.

Current Fabric runtime baseline: Fabric Runtime 2.0 is GA with Apache Spark 4.1, Python 3.13 and Delta Lake 4.2. Microsoft states that Runtime 2.0 is not yet the default as of this build date and plans the default rollout for late September 2026. Always record the runtime actually attached to your workspace/environment.

Important continuity: M13 used a current standalone/local reference baseline of Spark 4.2 + Delta 4.4. Fabric is a managed service with its own supported runtime bundle, so do not force M13 package versions into Fabric notebooks.

Before starting Complete M14 and G05. You need a Microsoft/Fabric account with an available Fabric capacity or trial for live portal work. If access is unavailable, study the full book, execute the local simulators, watch the visual sources and mark live Fabric evidence as pending rather than faking screenshots.

Video-first rule Useful legitimate videos are kept visible inside the matching lesson. If a publisher exposes reliable chapter boundaries, the exact range is printed. If not, the video remains as optional visual support with no invented timestamp. The learner book and supplied practice remain self-sufficient; current Microsoft documentation controls version-sensitive behavior.

Practice: 14_M15/practice/ Quick revision:

12_REVISION_AND_TESTS/M15/M15_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M15/PROTECTED_OPEN_AFTER_ATTEMPT/M15_Module_Revision_Test_Answers.pdf

DE15-01

Cloud fundamentals for data engineers

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

02:19:27-02:39:20 - architectural decision-making

[2. Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 1 of 16 | Primary practice: `practice/fabric_patterns/01_cloud_boundary.py`

Why this matters

Cloud data engineering is not simply 'running the same laptop code on somebody else's computer.' The cloud changes ownership boundaries, identity, billing, durability, network reachability and failure assumptions. A junior engineer who can draw those boundaries is safer than one who memorizes service names. In Fabric, much of the infrastructure is SaaS-managed, but you still own the data contract, access model, workload behavior and evidence that a run produced the right result.

Picture it in your head

Picture three layers of responsibility. Microsoft operates the physical platform and much of the service fabric. Your organization chooses capacity, tenant/workspace governance and identities. Your data team chooses items, schemas, pipelines, code, validation and promotion. When an incident happens, ask first which boundary owns the failed component instead of randomly changing settings.

Words before we use them

Word	Meaning before we use it
tenant	The organization-level identity and administration boundary.
capacity	A pool of Fabric compute resources measured in Capacity Units (CUs).
workspace	A collaboration/security container that holds Fabric items.
item	A Fabric object such as a lakehouse, notebook, pipeline or warehouse.
control plane	APIs/UI used to configure resources.
data plane	The path where actual data is read, written or processed.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course 02:19:27-02:39:20](#) - architectural decision-making

[VIDEO: Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#) Optional visual - no verified bounded range

[OFFICIAL/CURRENT: Microsoft Fabric Data Engineering overview](#)

[OFFICIAL/CURRENT: Fabric workspaces](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

A regional courier company wants one platform for parcel scan files, transformations and analytics. The beginner answer is 'put everything in Fabric.' The engineering answer decomposes it: a workspace contains the lakehouse, notebooks and pipeline; OneLake holds durable data; Fabric compute executes Spark/SQL/pipeline work; Entra identities decide who may operate the workspace; capacity determines how much shared compute is available. The source system remains an external dependency. If the source API is down, adding more Fabric capacity does not repair the source. If a notebook lacks permission, restarting Spark does not grant authorization.

Write those components in a table with columns: component, owner, durable state, identity used, observable evidence, recovery action. This is a simple exercise, but it is the beginning of production reasoning.

What you should see

You can point at every box in a small cloud data-flow diagram and say what it stores, what executes there, who authorizes it, what it costs broadly, and what proof would show that it worked.

Why it worked

The design separates service names from responsibilities. That lets you reason when the product UI changes because storage, compute, identity, orchestration and observation are still distinct concerns.

Break it / common failure

A common mistake is assuming SaaS means 'nothing to operate.' Fabric removes server patching and cluster provisioning work, but your team still owns bad joins, duplicate loads, over-broad access, runaway scans, broken dependencies and untested deployments. Another mistake is thinking a green pipeline run proves business correctness.

Reason through a realistic incident

A notebook is slow and the capacity dashboard shows throttling. At the same time, a source API is returning 429 responses. Do not collapse these into one vague 'cloud is slow' incident. Separate source throttling from Fabric capacity pressure, record which stage waits, and test one hypothesis at a time.

Deepen the engineering reasoning

Cloud architecture becomes easier when you name state. Configuration state lives in workspace/item definitions; durable business data lives in OneLake or external systems; transient compute state lives in running jobs; deployment state lives in Git/pipeline metadata; progress state might live in a watermark table. Recovery depends on restoring the correct state, not simply rerunning everything. Also distinguish scalability from elasticity: a service may scale to large workloads, while elasticity describes adjusting resources to demand. Finally, recognize cost as a correctness-adjacent constraint: a pipeline that produces the right table by scanning 50 TB every five minutes can still be an unacceptable production design.

Try it yourself

Draw a six-box architecture for orders arriving from an operational database into a Fabric lakehouse and then a gold table. Label storage, compute, identity and monitoring boundaries. Do not use more than two Fabric compute item types.

Fresh changed retry

Change the source from an Azure database to an on-premises database behind a firewall. Add only the new connectivity/gateway/private-access concern; keep the rest of the architecture stable.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Cloud DE = managed platform plus your explicit responsibilities for data, identity, orchestration, evidence and cost. Diagnose by boundary, not by brand name.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-02

Identity, IAM and least privilege

Watch first

[1. CodeLucky - Azure RBAC Explained: Role-Based Access Control Tutorial for Beginners](#)

00:26-04:18 - RBAC core components through best practices

[2. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

01:22:30-01:50:53 - security and governance

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 2 of 16 | Primary practice: `practice/fabric_patterns/02_rbac_scope_simulator.py`

Why this matters

Most serious cloud data incidents are easier to create with permissions than with Python. Identity and access management (IAM) determines who or what can create resources, read data, run pipelines and retrieve secrets. Least privilege is not a slogan: it is the discipline of granting the smallest practical permission at the smallest practical scope for the shortest practical time.

Picture it in your head

Picture a keyring. Identity answers 'whose keyring is this?' Authentication proves that identity. Authorization checks whether a specific key opens a specific door. Azure RBAC adds three pieces to every permission decision: principal, role and scope. Fabric workspace roles add another collaboration layer inside Fabric.

Words before we use them

Word	Meaning before we use it
principal	A user, group, service principal or managed identity requesting access.
role definition	A named set of allowed actions.
scope	The resource boundary where a role assignment applies.
authentication	Proving an identity.
authorization	Deciding what that identity may do.
least privilege	Granting only the access required for the job.

Useful video / source route

[VIDEO: CodeLucky - Azure RBAC Explained: Role-Based Access Control Tutorial for Beginners](#)

00:26-04:18 - RBAC core components through best practices

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 01:22:30-01:50:53 - security and governance

[OFFICIAL/CURRENT: Azure RBAC overview](#)

[OFFICIAL/CURRENT: Fabric workspace roles](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Suppose Priya develops a notebook that reads a bronze lakehouse and writes silver tables. She needs edit/run access in the development workspace, but an analyst who consumes gold output may only need Viewer/read permissions. A deployment service principal may need permission to update supported Fabric items without needing interactive human access.

In the supplied simulator, create three assignments: Analyst -> Viewer at workspace DEV; Engineer -> Contributor at workspace DEV; DeploymentSP -> Contributor at workspace TEST. Then ask whether Analyst can delete a notebook, whether Engineer can edit it, and whether DeploymentSP can modify TEST. The result should follow from principal + role + scope, not from the person's job title.

What you should see

You can explain why 'Contributor everywhere' is dangerous, distinguish workspace roles from Azure resource roles, and reason about effective permissions without guessing.

Why it worked

Permissions become reviewable when each assignment has a concrete principal, role and scope. Least privilege is impossible to verify when access is granted through vague shared accounts or undocumented owner-level permissions.

Break it / common failure

Two beginner failures recur. First, giving a human account broad Owner/Admin access because a lab is blocked. Second, confusing Microsoft Entra directory roles, Azure RBAC roles and Fabric workspace roles. They interact with different resource planes. Fix the missing permission at the layer that actually denies the action.

Reason through a realistic incident

A pipeline worked yesterday under your personal account but fails when scheduled under a service identity. That is valuable evidence: the code path is probably valid, while the runtime identity lacks some required permission or connection. Compare effective identities and scopes before editing transformation logic.

Deepen the engineering reasoning

Machine identities deserve the same rigor as users. Prefer managed identities or federated/service identities over embedding long-lived secrets when the platform supports them. Separate human development permissions from production execution identity. Use groups where possible so access follows job function instead of dozens of one-off grants. Remember that RBAC is usually additive: multiple assignments can combine into broader effective access. The review question is not only 'can this identity run?' but also 'what else could this identity do if compromised?'

Try it yourself

Design roles for four personas: developer, data consumer, deployment automation and production support. State the minimum workspace-level role and one thing each persona must not be allowed to do.

Fresh changed retry

A contractor joins for two weeks to troubleshoot a read-only gold-table issue. Redesign your assignment so the contractor cannot edit notebooks, pipelines or security configuration.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Access decision = principal + role + scope. Keep identities separate, privileges narrow and production execution independent of personal accounts.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-03

Azure Storage concepts and OneLake

Watch first

[1. Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#)

Optional visual - no verified bounded range

[2. endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#)

02:10-04:43 - Fabric lakehouse and OneLake table/file structure

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 3 of 16 | Primary practice: `practice/fabric_patterns/03_storage_onelake_paths.py`

Why this matters

Before Fabric, Azure data engineers often assembled storage accounts, ADLS Gen2 containers, compute services and orchestration separately. OneLake keeps familiar data-lake ideas but presents one logical lake for the Fabric tenant. You still need to understand files, folders, table areas, URIs and external storage because shortcuts and interoperability depend on those boundaries.

Picture it in your head

Think of ADLS Gen2 as a set of storage accounts and containers you explicitly provision. Think of OneLake as Fabric's tenant-wide logical lake where each Fabric item provisions its own storage view. A lakehouse gives you a Files area for arbitrary files and a Tables area for managed/queryable table data.

Words before we use them

Word	Meaning before we use it
object storage	Durable storage addressed by container/path rather than disk blocks.
ADLS Gen2	Azure Data Lake Storage capabilities built on Blob Storage with hierarchical namespace.
OneLake	Fabric tenant-wide logical data lake.
Files area	Lakehouse location for arbitrary raw/semi-structured files.
Tables area	Lakehouse location intended for managed table data, normally Delta.
ABFS URI	Azure Blob FileSystem-style path that Spark-compatible engines can use.

Useful video / source route

[VIDEO: Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#) Optional visual - no verified bounded range

[VIDEO: endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#) 02:10-04:43 - Fabric lakehouse and OneLake table/file structure

[OFFICIAL/CURRENT: Get data into OneLake](#)

[OFFICIAL/CURRENT: Connect to OneLake / ABFS paths](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Create a path inventory for a lakehouse named ops_lh in workspace de_dev. Raw CSV lands under Files/bronze/orders/2026/09/14/. A curated Delta table is exposed under Tables/silver_orders. Write the human-readable logical path and the ABFS-style shape. Do not invent a local C: drive path; OneLake is remote service storage.

The practice script validates that a path has a workspace, item name/type and Files/Tables boundary. It does not pretend to authenticate to OneLake. That separation is intentional: path reasoning can be tested locally; actual Entra-authenticated access remains learner-PC/Fabric proof.

```
abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<lakehouse>.lakehouse/Files/bronze/orders/
```

What you should see

You can decide whether a dataset belongs in Files or Tables, read a OneLake path without panic, and explain how OneLake differs from but interoperates with Azure storage concepts.

Why it worked

Storage design became explicit: raw file landing and managed tables are separate contracts even though both live in OneLake. The path itself records enough context to avoid mysterious hard-coded mounts.

Break it / common failure

Do not assume anything placed under Files automatically appears as a SQL-queryable table. Conversely, do not treat the Tables area as a dumping ground for random CSVs. Another mistake is hard-coding workspace/item display names into production code without a configuration strategy for DEV/TEST/PROD.

Reason through a realistic incident

A notebook can read a file path but the SQL analytics endpoint cannot see it as a table. That is not automatically a permission bug. Check whether the data is actually represented as a supported Delta table in the lakehouse Tables area or exposed through an appropriate shortcut.

Deepen the engineering reasoning

Cloud storage paths are interfaces. Renaming a workspace or item, moving from DEV to PROD, or changing an external source can break brittle strings. Prefer parameterized logical references, environment configuration and documented ownership. Also distinguish physical copy from logical reference: OneLake shortcuts can expose data without duplicating it, which changes freshness, security and failure reasoning. Finally, understand that storage durability does not replace data-quality controls; a perfectly durable bad file is still bad data.

Try it yourself

Design a Files/Tables layout for raw CRM JSON, cleaned customer Delta data and a curated customer_daily table. State why each location is chosen.

Fresh changed retry

Move the raw CRM files to an external ADLS Gen2 account and use Fabric only for silver/gold. Explain what path/reference and identity assumptions change.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

OneLake is Fabric's logical lake; lakehouse Files stores arbitrary files and Tables exposes managed table data. Treat paths and environment bindings as explicit interfaces.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-04

Fabric workspace and item model

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

00:18:18-00:48:26 - admin settings and workspace governance

[2. Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 4 of 16 | Primary practice: practice/fabric_patterns/04_workspace_inventory.json

Why this matters

Fabric feels simple because many workloads share one portal, but that simplicity can hide lifecycle boundaries. A workspace is not just a folder. It is a collaboration, security, capacity and deployment boundary that contains items such as lakehouses, notebooks, pipelines, warehouses and environments.

Picture it in your head

Imagine a workshop. The workspace is the locked room. Items are the machines and workbenches inside it. Capacity is the shared electricity supply. People receive room-level roles, while some items may also have finer permissions. Moving a machine to another room changes which room policies and collaborators surround it.

Words before we use them

Word	Meaning before we use it
workspace	Container for Fabric items, collaboration and many security/lifecycle settings.
item	A first-class Fabric artifact such as Lakehouse, Notebook or Data pipeline.
workspace role	Admin, Member, Contributor or Viewer access boundary.
capacity assignment	Which Fabric capacity supplies compute for the workspace.
environment	A Fabric item for Spark runtime/libraries/compute configuration.
dependency	A reference from one item to another, such as notebook -> lakehouse.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 00:18:18-00:48:26 - admin settings and workspace governance

[VIDEO: Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#) Optional visual - no verified bounded range

[OFFICIAL/CURRENT: Fabric workspaces](#)

[OFFICIAL/CURRENT: Workspace roles](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Build a DEV workspace inventory with six items: bronze lakehouse, curated lakehouse, ingest pipeline, transform notebook, Fabric environment and validation notebook. For each item record owner, upstream dependencies, downstream consumers and whether its definition should be version controlled.

Then compare two architectures: one giant workspace containing DEV/TEST/PROD suffixes versus separate workspaces per environment. The second usually gives clearer promotion and access boundaries. The lesson is not that one pattern is universally correct; it is that environment boundaries should be deliberate.

What you should see

You can inspect a Fabric workspace and explain what each item does, what it depends on and why workspace placement matters for security and deployment.

Why it worked

An item inventory turns the portal from a collection of icons into a dependency graph. That makes missing bindings and unsafe environment mixing visible before deployment.

Break it / common failure

A common anti-pattern is storing experimental, production and personal artifacts in one workspace because it is convenient. Another is assuming a workspace role automatically grants every data-plane permission in every connected external system.

Reason through a realistic incident

A notebook deploys successfully to TEST but opens with a broken default lakehouse reference. The notebook file exists, so deployment 'succeeded,' yet the dependency binding is wrong. Inspect logical/physical item bindings instead of rerunning the same deployment.

Deepen the engineering reasoning

Workspace design influences blast radius. Admin rights on a small DEV workspace are very different from Admin rights on a shared production domain. Use naming conventions, folders/task flows and ownership metadata so a new engineer can understand the topology without tribal knowledge. Keep environment-specific values out of code where lifecycle tooling can bind or parameterize them. Treat dependencies as release inputs: a notebook, its environment, target lakehouse and required connection should be reviewed together.

Try it yourself

Create a proposed DEV workspace inventory for an order pipeline with at least one lakehouse, pipeline, notebook and environment. Draw dependency arrows.

Fresh changed retry

Split the design into DEV and PROD workspaces. Identify which dependencies must be rebound or configured differently after promotion.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Workspace = lifecycle/security/collaboration boundary; item = deployable Fabric artifact. Inventory dependencies before you deploy.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-05

OneLake paths, files, tables and shortcuts

Watch first

[1. Centida Consulting - Microsoft Fabric: OneLake Shortcuts](#)

02:29-03:17 - shortcut concept and hands-on creation

[2. endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#)

04:43-05:19 - table/file structure context before medallion

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 5 of 16 | Primary practice: `practice/fabric_patterns/05_shortcut_decision.py`

Why this matters

Shortcuts are one of Fabric's most important 'do not copy unless you must' features. They let OneLake expose data that physically lives elsewhere or in another Fabric item. This can reduce duplicate storage and simplify access, but it also means freshness, authorization and availability depend on the referenced source.

Picture it in your head

A shortcut is like a sign in one warehouse aisle that says 'the boxes are actually stored in Building B; follow this address.' The local directory looks convenient, but there is no second independent copy. If Building B denies access or goes down, the sign cannot manufacture the boxes.

Words before we use them

Word	Meaning before we use it
shortcut	A OneLake reference to data stored at another supported location.
materialized copy	A physically duplicated dataset under your control.
source of truth	The authoritative dataset whose changes drive downstream data.
delegated access	Access whose success depends on identity/permissions at another boundary.
staleness	How far a viewed dataset lags the authoritative source.
external location	Storage outside the current Fabric item/workspace.

Useful video / source route

[VIDEO: Centida Consulting - Microsoft Fabric: OneLake Shortcuts](#) 02:29-03:17 - shortcut concept and hands-on creation

[VIDEO: endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#) 04:43-05:19 - table/file structure context before medallion

[OFFICIAL/CURRENT: OneLake shortcuts](#)

[OFFICIAL/CURRENT: OneLake quickstart with shortcut reuse](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

A sales team already has curated Delta data in a central lakehouse. Finance wants to query the same data. Option A copies 200 GB nightly into a finance lakehouse. Option B creates a OneLake shortcut to the curated table/folder. If Finance only needs read access and the source ownership/SLA are acceptable, the shortcut may avoid duplication and preserve one source of truth. If Finance needs an independently frozen month-end snapshot, a shortcut alone may not satisfy the requirement.

Run the shortcut decision script with fields `copy_required`, `independent_retention`, `source_sla_acceptable` and `source_format_delta`. The output is a recommendation plus reasons, not a magical product rule.

What you should see

You can explain when a shortcut is elegant and when a physical copy is justified, including the security and availability trade-off.

Why it worked

The decision starts from business guarantees - independent retention, freshness, ownership - rather than choosing shortcuts just because they are new.

Break it / common failure

Do not promise independent disaster recovery from a shortcut to the same upstream data. Do not assume a shortcut changes the underlying file format into Delta. And do not forget that consumers still need permissions appropriate to the path through which they access the data.

Reason through a realistic incident

A table exposed through a shortcut suddenly fails for one team while the source owner can still read it. Compare the consuming identity, shortcut configuration, source permissions and source health. Copying the data immediately may hide the real authorization problem and create uncontrolled duplicates.

Deepen the engineering reasoning

A shortcut is a data-contract decision. Document source owner, expected freshness, schema-change process, retention and support contact. When you build medallion layers across domains, shortcuts can provide governed access to existing data products without centralizing every byte. But that decoupling works only if ownership is explicit. Also consider performance: remote/reference access patterns can behave differently from local managed tables, so measure important queries rather than promising zero-copy is always faster.

Try it yourself

For three datasets - daily exchange rates, immutable regulatory snapshot, and a 4-TB shared customer dimension - choose shortcut or copy and defend the decision.

Fresh changed retry

Change the customer dimension requirement so PROD must continue operating for four hours even if the source workspace is unavailable. Revisit your decision.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Shortcut = reference, not independent copy. Decide from ownership, freshness, retention, security and availability requirements.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

Keep Fabric and Databricks terms separate

OneLake and its shortcuts are part of the Fabric route taught here. Unity Catalog is Databricks governance; DBFS mount examples belong to a Databricks environment. Do not copy a mount-based tutorial into a Fabric notebook and assume the storage or permission model is the same. An integration between platforms must name both sides and its access boundary.

Reference: <https://docs.databricks.com/aws/en/data-governance/unity-catalog/>.

DE15-06

Fabric Lakehouse and Delta tables

Watch first

[1. endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#)

00:22-04:43 - lakehouse concept, Fabric lakehouse, files/tables

[2. Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 6 of 16 | Primary practice: `practice/fabric_patterns/06_lakehouse_delta_patterns.sql`

Why this matters

M13 taught Delta mechanics locally. M15 moves that reasoning into Fabric's managed lakehouse. The important transition is recognizing that OneLake storage, Spark notebooks and the SQL analytics endpoint can operate over the same Delta-backed lakehouse without you designing a separate storage system for each engine.

Picture it in your head

Picture one set of Delta table files in OneLake. Spark is the engineering workbench that can write and transform them. The SQL analytics endpoint is a read-oriented T-SQL window over supported lakehouse Delta tables. Different doors, same underlying lakehouse table state.

Words before we use them

Word	Meaning before we use it
lakehouse	Fabric item combining lake-style files with managed analytic tables.
Delta table	Table represented by Parquet data plus Delta transaction log metadata.
managed table	Table whose storage/location lifecycle is managed by the lakehouse/catalog.
SQL analytics endpoint	Automatically provisioned T-SQL query surface over lakehouse Delta tables.
Spark engine	Distributed compute used to read/write/transform lakehouse data.
table contract	Expected grain, keys, schema and quality guarantees.

Useful video / source route

[VIDEO: endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#) 00:22-04:43 - lakehouse concept, Fabric lakehouse, files/tables

[VIDEO: Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#) Optional visual - no verified bounded range

[OFFICIAL/CURRENT: Lakehouse end-to-end scenario](#)

[OFFICIAL/CURRENT: SQL analytics endpoint](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Use the earlier orders domain. Raw CSV lands under Files/bronze/orders. A Fabric notebook reads it, validates schema, and writes a Delta table Tables/silver_orders. A gold table aggregates daily sales. Spark owns the transformation/write path. The SQL analytics endpoint exposes the Delta tables for read-oriented T-SQL analytics.

The supplied SQL file contains expected read queries and boundary comments. It intentionally does not pretend you can perform Spark-style Delta writes through the read-only lakehouse SQL endpoint. That distinction matters in interviews and real designs.

```
# Fabric notebook concept
bronze = spark.read.option('header', True).csv('Files/bronze/orders/*.csv')
bronze.write.format('delta').mode('append').saveAsTable('silver_orders')
```

What you should see

You can say exactly which engine writes the table, which surface queries it, and where the durable data lives. You do not call every Fabric SQL surface a warehouse.

Why it worked

The design preserves one durable table contract while allowing multiple query/processing experiences. That reduces unnecessary data copies and keeps reconciliation centered on one Delta state.

Break it / common failure

A frequent mistake is trying to INSERT/UPDATE lakehouse Delta tables through the SQL analytics endpoint as if it were a normal writable SQL database. Another is assuming an arbitrary external Delta location will automatically appear in the endpoint without supported registration/shortcut behavior.

Reason through a realistic incident

Spark shows 1,000 new rows but a downstream SQL consumer does not immediately see what you expect. Verify that Spark wrote a valid lakehouse Delta table in the Tables area, inspect endpoint metadata/synchronization state, and compare counts by a stable boundary. Do not rewrite data merely to force visibility unless the root cause is understood.

Deepen the engineering reasoning

The lakehouse is a convergence point, not an excuse to abandon modeling discipline. Delta ACID protects table transactions, but it does not choose the correct grain or business key. Schema evolution is a tool, not permission to accept uncontrolled drift. Maintain raw evidence, explicit transformation stages and reconciliation. Also note the runtime distinction: M13's local/reference Spark 4.2 + Delta 4.4 baseline is not the same thing as Fabric's service-managed runtime. In Fabric Runtime 2.0, the current GA combination is Spark 4.1 + Delta 4.2; write code against the actual service runtime you are using.

Try it yourself

Design Files and Tables areas for orders, customers and daily_sales. State which objects Spark writes and which are queried from the SQL endpoint.

Fresh changed retry

A consumer needs writable T-SQL transformations and stored procedures on the curated layer. Explain when a Fabric Warehouse becomes a better serving/transform choice than relying only on the lakehouse endpoint.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Fabric Lakehouse = OneLake files + Delta tables + Spark engineering + an automatically provisioned read-oriented SQL analytics endpoint.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-07

Fabric notebooks

Watch first

[1. Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#)

04:31-25:03 - notebook overview; 25:03-41:38 - NotebookUtils

[2. endjin - Microsoft Fabric: Good Notebook Development Practices](#)

00:12-13:05 - notebook structure, modularization and schemas as code

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 7 of 16 | Primary practice: `practice/fabric_patterns/07_notebook_patterns.py`

Why this matters

A notebook is useful because code, explanation and results can sit together. That convenience becomes dangerous when a production pipeline depends on a 600-cell document with hidden state and manual clicks. Good Fabric notebooks behave like reviewable programs: explicit inputs, small functions, parameters, deterministic outputs, validation and a clear execution path.

Picture it in your head

Treat a notebook as a thin orchestration shell around reusable logic, not as a scratchpad that accidentally became production. Markdown explains the contract; parameter cells define inputs; functions/modules hold repeated logic; validation cells prove outputs; the final cell returns a clear status/value.

Words before we use them

Word	Meaning before we use it
cell state	Variables/results created by prior cell execution.
parameterization	Supplying environment/run values instead of hard-coding them.
NotebookUtils	Fabric utility APIs for files, credentials, notebook orchestration and other platform tasks.
idempotent cell	A cell safe to execute again without corrupting results.
default lakehouse	Lakehouse attached to the notebook for convenient relative access.
environment	Versioned/configurable runtime and library settings associated with Spark workloads.

Useful video / source route

[VIDEO: Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#) 04:31-25:03 - notebook overview; 25:03-41:38 - NotebookUtils

[VIDEO: endjin - Microsoft Fabric: Good Notebook Development Practices](#) 00:12-13:05 - notebook structure, modularization and schemas as code

[OFFICIAL/CURRENT: Fabric Data Engineering docs - notebooks](#)

[OFFICIAL/CURRENT: Notebook credentials utilities](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

The sample notebook pattern has five phases: parameters -> load -> validate -> transform/write -> reconcile/exit. A run_date parameter and source path are read once. The transformation function accepts data rather than reaching into global state. Before writing, the notebook checks required columns and rejects duplicates on the business key. After writing, it records row counts and a reconciliation status.

Open 07_notebook_patterns.py as ordinary Python to inspect the structure. In Fabric, translate the same shape into notebook cells. The goal is not to make local Python pretend to be Fabric; it is to practice the code organization before the cloud UI adds another layer.

```
RUN_DATE = '2026-09-14'    # pipeline parameter in real run
TARGET_LAYER = 'silver'
assert TARGET_LAYER in {'bronze', 'silver', 'gold'}
```

What you should see

You can open a notebook and quickly locate parameters, input contract, transformation logic, output write, validation and failure/recovery evidence.

Why it worked

Explicit phases reduce hidden state. The notebook can be scheduled or called from a pipeline because it no longer relies on a human remembering which cells to run in which order.

Break it / common failure

Classic notebook failures include running cells out of order, leaving stale variables in memory, hard-coding PROD paths, installing libraries ad hoc in one session, printing secrets, and mixing exploratory display code with irreversible writes.

Reason through a realistic incident

A scheduled notebook fails but an engineer says 'it works when I open it and run the last three cells.' That is evidence of hidden state or missing initialization. Restart the session and execute the documented path from the top with explicit parameters; a production notebook must not depend on yesterday's interactive state.

Deepen the engineering reasoning

A notebook can still be production-grade when its boundaries are disciplined. Put environment/library configuration in a Fabric Environment where appropriate. Use Git-friendly notebook source control and keep outputs out of versioned source. Separate pure transformation functions from Fabric I/O so important logic can be unit-tested more easily. Return structured status rather than only printing messages. Treat notebook-to-notebook calls as dependencies that need error propagation and parameter contracts, not as a substitute for modular code.

Try it yourself

Refactor a pseudo-notebook with hard-coded source path, secret and run date into parameterized sections. Write the validation checks that must happen before any write.

Fresh changed retry

Change the notebook so a pipeline can pass `run_date` and `target_layer`. Add a safe failure response when `target_layer` is not bronze/silver/gold.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Production notebook = explicit parameters + deterministic execution + modular logic + validation + reconcilable output + no secret/hard-coded environment leakage.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-08

Spark in Fabric

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

03:12:43-03:30:05 - hands-on PySpark

[2. Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#)

54:23-01:00:37 - Spark SQL in Fabric notebooks

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 8 of 16 | Primary practice: `practice/fabric_patterns/08_spark_runtime_probe.py`

Why this matters

You already know Spark concepts from M12. The cloud lesson is how Fabric supplies and governs Spark: runtime versions, environments, session startup, libraries, capacity consumption and built-in monitoring. Code that works under local Spark 4.2 can behave differently under a managed Fabric runtime if APIs or dependencies differ.

Picture it in your head

Think of your notebook code as the passenger and the Fabric runtime as the train. You choose a supported train/runtime and environment; Fabric provisions the Spark session and infrastructure. You must still inspect the train number before assuming which language/library features are available.

Words before we use them

Word	Meaning before we use it
Fabric Runtime	Microsoft-managed Spark runtime bundle with specific Spark/Python/Delta versions.
environment item	Fabric item that can pin runtime, libraries and Spark configuration.
session	A running Spark application context for notebook work.
native execution engine	Fabric optimization path for supported Spark workloads.
library dependency	Package required by notebook/Spark code.
runtime drift	Behavior change caused by version/environment differences.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 03:12:43-03:30:05 - hands-on PySpark

[VIDEO: Aleksii Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#) 54:23-01:00:37 - Spark SQL in Fabric notebooks

[OFFICIAL/CURRENT: Fabric Runtime 2.0](#)

[OFFICIAL/CURRENT: Spark monitoring overview](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

As of this course build, Fabric Runtime 2.0 is GA with Apache Spark 4.1, Python 3.13 and Delta Lake 4.2. Microsoft states that it is not yet the default runtime and plans the default rollout for late September 2026. Therefore, the first cell of a serious lab records runtime/version evidence rather than assuming it.

The supplied runtime probe prints placeholders locally and documents the Fabric-side commands you should record: Spark version, Python version, attached environment and one tiny DataFrame action. Save that evidence with the lesson. Then compare it with M13's standalone/local baseline and explain why the numbers need not match.

```
print(spark.version)
import sys
print(sys.version)
# Also record the Fabric Runtime / attached Environment shown in the portal.
```

What you should see

You can identify the active Fabric runtime and explain why environment/runtime pinning matters for reproducibility and deployment.

Why it worked

The runtime becomes part of the evidence. When behavior changes after promotion or a platform update, you have something concrete to compare instead of saying 'Spark suddenly broke.'

Break it / common failure

Do not pip-install random incompatible versions in a notebook just to force parity with a laptop. Do not assume the newest runtime is automatically default in every workspace. Do not diagnose every slow Spark job as a capacity problem without checking plan, data shape and shuffle behavior from M12.

Reason through a realistic incident

DEV works after an engineer manually installs a library in-session; PROD fails after deployment. The code commit is identical. Inspect environment/library configuration: the dependency lived only in interactive session state, not in a deployable environment definition.

Deepen the engineering reasoning

Managed Spark does not remove distributed-systems reasoning. Partitions, shuffles, skew, caching and driver safety from M12 still matter. Fabric adds lifecycle concerns: runtime deprecation, environment publication, capacity/queueing and service monitoring. Keep a small compatibility smoke test for critical jobs: create session, read a known tiny table, run one transformation, write to a disposable location, reconcile count, and clean up. Run it when runtime or environment changes are promoted.

Try it yourself

Write a runtime evidence checklist for DEV and PROD. Include at least Spark, Python, Delta/runtime, attached environment and one library version your project depends on.

Fresh changed retry

Assume PROD remains on an older Fabric runtime while DEV opts into Runtime 2.0. List three tests you would require before promoting a notebook that uses a new Spark/Python feature.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Fabric Spark is managed, not versionless. Record runtime evidence, version environments and carry M12 distributed-processing discipline into the cloud.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-09

Fabric Data Pipelines

Watch first

[1. Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#)

04:49-27:35 - pipeline overview and Copy Data activity

[2. Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)

02:00-12:30 - loading lakehouse data with a pipeline and schema/type handling

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 9 of 16 | Primary practice:
practice/fabric_patterns/09_pipeline_incremental_simulator.py

Why this matters

Pipelines coordinate work across connectors, copy activities, notebooks, procedures and schedules. The most important skill is not dragging boxes onto a canvas; it is designing dependencies, parameters, retries and progress state so a partial failure can be understood and safely replayed.

Picture it in your head

A pipeline is a conductor, not the orchestra. Copy activity moves data, a notebook transforms it, a validation step checks it, and a progress update records what is safe to consider committed. The conductor decides order and failure branches; each activity still has its own contract.

Words before we use them

Word	Meaning before we use it
activity	One unit of work in a pipeline.
dependency condition	Rule controlling whether a downstream activity runs after success/failure/completion.
parameter	Value supplied at run time to make a pipeline reusable.
dynamic content	Expressions that derive activity inputs from parameters or prior outputs.
retry	Policy to repeat a failed activity under controlled conditions.
run ID	Identifier for one pipeline execution used for tracing evidence.

Useful video / source route

[VIDEO: Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#) 04:49-27:35
- pipeline overview and Copy Data activity

[VIDEO: Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)
02:00-12:30 - loading lakehouse data with a pipeline and schema/type handling

[OFFICIAL/CURRENT: Fabric Copy activity](#)

[OFFICIAL/CURRENT: Incremental load tutorial](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Build the logical pipeline `lookup\_old\_watermark -> lookup\_new\_watermark -> copy\_increment -> validate -> commit\_watermark`. The critical rule is the same pattern repeated in M08: do not commit progress before the copied/validated result is durable. If validation fails, the old watermark must remain so replay can re-read the failed interval.

Run 09_pipeline_incremental_simulator.py locally. First it simulates a successful copy and commits the candidate watermark. Then it simulates a validation failure and proves that the committed watermark does not advance. This is a cloud-pipeline lesson grounded in a local deterministic control.

```
old = committed_watermark()
candidate = source_max_watermark()
rows = copy_interval(old, candidate)
validate(rows)
write_durable(rows)
commit_watermark(candidate) # only after durable validation
```

What you should see

You can read a Fabric pipeline and identify parameter flow, durable state, failure branches and the exact step that commits incremental progress.

Why it worked

Separating candidate progress from committed progress makes retries predictable. A visual green arrow is not the contract; the state transition is.

Break it / common failure

Common failures: updating watermark immediately after source read, using one giant activity with hidden side effects, swallowing notebook failure and still running the commit step, or retrying a non-idempotent external side effect without a deduplication key.

Reason through a realistic incident

The Copy activity succeeded, the notebook failed halfway through, and the next scheduled run finds no new rows. Investigate whether progress was committed too early. Restore the last safe watermark from evidence rather than manually increasing the source filter until rows 'look right.'

Deepen the engineering reasoning

Pipelines should make control flow observable. Name activities by intent, not Activity1. Pass run_id and logical date into downstream steps so logs reconcile. Set retry only for failures that can safely be retried. Use failure paths to persist diagnostics, not to hide the failure. For large systems, metadata-driven patterns can reduce repeated pipelines, but abstraction is useful only after you can debug one concrete path. Always preserve one simple end-to-end lineage from source boundary to durable target and progress marker.

Try it yourself

Design a pipeline for daily customer changes with source lookup, copy, transform, validate and commit. Mark which activities are safe to retry and why.

Fresh changed retry

Add a transient source timeout after the new watermark is read but before copy. Explain what may be retried without changing committed state.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Pipeline orchestration is state management. Commit progress only after durable validated output; parameterize runs and make retry semantics explicit.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-10

SQL analytics endpoint and warehouse choice

Watch first

[1. TechBrothersIT - Understanding SQL Analytics Endpoint in Microsoft Fabric](#)

Optional visual - no verified bounded range

[2. Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)

12:30-18:30 - analyze lakehouse data and downstream reporting context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 10 of 16 | Primary practice: `practice/fabric_patterns/10_sql_endpoint_examples.sql`

Why this matters

Fabric offers multiple SQL-shaped experiences, so beginners often choose by familiarity instead of workload. Every lakehouse gets a SQL analytics endpoint that provides a read-oriented T-SQL surface over its Delta tables. A Fabric Warehouse is a different item intended for warehouse-style SQL engineering and writable T-SQL scenarios.

Picture it in your head

The lakehouse SQL endpoint is a window looking into lakehouse Delta tables. You can query and build SQL-layer objects allowed by the endpoint, but the underlying engineering writes remain in the lakehouse/Spark path. A Warehouse is a SQL-first workshop where T-SQL data engineering and warehousing are first-class.

Words before we use them

Word	Meaning before we use it
T-SQL	Microsoft SQL dialect used by Fabric SQL experiences.
SQL analytics endpoint	Automatically created SQL query surface for a lakehouse.
read-only table access	Ability to query lakehouse table data without using the endpoint to mutate it.
Warehouse	Fabric SQL-first data warehouse item.
serving layer	Curated interface consumed by BI/analysts/applications.
cross-database query	Query that references data across supported database/endpoint boundaries.

Useful video / source route

[VIDEO: TechBrothersIT - Understanding SQL Analytics Endpoint in Microsoft Fabric](#) Optional visual
- no verified bounded range

[VIDEO: Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)
12:30-18:30 - analyze lakehouse data and downstream reporting context

[OFFICIAL/CURRENT: SQL analytics endpoint](#)

[OFFICIAL/CURRENT: Lakehouse SQL endpoint use cases](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Your silver and gold Delta tables are produced by Spark in a lakehouse. Analysts want T-SQL SELECT, views and a familiar connection endpoint. The SQL analytics endpoint fits. A separate finance workload requires SQL-first DDL/DML, stored procedures and curated dimensional transformations that the team wants to own in T-SQL. That requirement may justify a Fabric Warehouse for the gold serving layer.

Use 10_sql_endpoint_examples.sql to label each statement READ, ENDPOINT-OBJECT, or WRITE-TO-TABLE. The exercise is conceptual: statements that mutate lakehouse Delta table data through the endpoint are marked as the wrong surface.

```
-- Lakehouse SQL analytics endpoint: read-oriented over Delta tables
SELECT TOP 100 order_id, order_date, amount
FROM dbo.gold_orders
ORDER BY order_date DESC;
```

What you should see

You can choose lakehouse endpoint versus Warehouse based on write model, engine preference and serving requirements, not merely because both show a SQL editor.

Why it worked

The choice is anchored to ownership of table writes. The same data platform can use Spark for lakehouse engineering and SQL for serving without pretending every SQL surface has identical capabilities.

Break it / common failure

Do not teach 'SQL endpoint = warehouse.' Do not promise full T-SQL DML against lakehouse Delta tables through the endpoint. Also do not create copies into Warehouse by default when the lakehouse endpoint already satisfies read analytics requirements.

Reason through a realistic incident

A developer tries to run UPDATE on a lakehouse Delta table through the SQL endpoint and gets blocked. The correct recovery is to use an appropriate Spark/Delta write path or reconsider whether the dataset should be owned by a Warehouse - not to disable security or export/re-import the table manually.

Deepen the engineering reasoning

Serving design should minimize duplicated truth. If a lakehouse gold layer already meets performance and semantic needs, the endpoint can be enough. If SQL-first teams require controlled DML, stored procedures, dimensional warehouse patterns or different performance/governance semantics, Warehouse can be appropriate. Measure critical query patterns and document where each table is authoritative. OneLake allows lakehouse and warehouse architectures to coexist, but coexistence without ownership rules becomes another form of data sprawl.

Try it yourself

Classify four needs: ad-hoc SELECT over Delta, Spark MERGE, stored-procedure-based SQL transformation, and Power BI consumption. Choose the appropriate Fabric surface for each.

Fresh changed retry

The BI team now needs a writable SQL staging layer before the final star schema. Redesign without creating two competing authoritative gold tables.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Lakehouse SQL endpoint = read-oriented T-SQL window over lakehouse Delta tables; Warehouse = SQL-first writable warehousing item. Choose by ownership/write contract.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

Where dbt fits in Fabric — platform awareness

The M09 project already teaches dbt models, sources, ref(), tests, materializations and documentation on a free local engine. Fabric can also run managed dbt jobs. As checked on 15 September 2026, Microsoft labels that feature preview. It requires tenant enablement and suitable workspace/data permissions. It is an optional platform translation here, not a replacement for your local dbt practice or a reason to buy capacity.

Keep the execution route clear. dbt-fabric targets the Fabric Warehouse. For a Lakehouse-only pattern, dbt-fabricspark executes Spark SQL through Fabric Livy and writes Delta tables in OneLake. The Lakehouse SQL analytics endpoint remains read-only; pointing a write-oriented model at that endpoint does not make it writable. Fabric pipelines can orchestrate the dbt job while dbt owns its model graph and tests. Adapter capabilities and SQL dialects differ.

The managed dbt job runtime is a separate environment from the Fabric Spark runtime. Do not infer its Python or dbt version from the words “Fabric Runtime 2.0”. Use each feature’s own compatibility matrix. Official references:

<https://learn.microsoft.com/en-us/fabric/data-factory/dbt-job-overview> and
<https://learn.microsoft.com/en-us/fabric/data-factory/common-dbt-job-patterns> .

DE15-11

Medallion architecture in Fabric

Watch first

[1. Microsoft Power BI - Learn Together: Organize a Fabric lakehouse using medallion architecture design](#)

09:55-42:55 - medallion principles, formats, movement and Fabric implementation

[2. endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#)

05:19-09:50 - medallion rationale through replayability/idempotency

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 11 of 16 | Primary practice: `practice/fabric_patterns/11_medallion_contracts.py`

Why this matters

Bronze/Silver/Gold is useful only when each layer has a clear contract. Fabric now explicitly recommends medallion as a lakehouse organization pattern, but naming folders bronze and silver does not create quality. The engineering value is progressive trust: raw evidence is preserved, validated/conformed data is separated, and consumption-ready models have stable business semantics.

Picture it in your head

Picture three inspection stations. Bronze receives the sealed shipment and records what arrived. Silver opens it, validates labels, removes known defects and standardizes units. Gold packages trusted products for a specific consumer. If Gold is wrong, you should be able to trace back through Silver to the original Bronze evidence.

Words before we use them

Word	Meaning before we use it
bronze	Raw or near-raw ingestion layer that preserves source evidence.
silver	Validated, standardized and deduplicated/conformed layer.
gold	Consumption-oriented curated layer with business-ready semantics.
contract	Explicit grain/schema/quality/ownership promise for a dataset.
replayability	Ability to rebuild downstream layers from retained upstream evidence.
quarantine	Separate durable location for records that fail validation.

Useful video / source route

[VIDEO: Microsoft Power BI - Learn Together: Organize a Fabric lakehouse using medallion architecture design](#) 09:55-42:55 - medallion principles, formats, movement and Fabric implementation

[VIDEO: endjin - Microsoft Fabric: Lakehouse & Medallion Architecture - Part 1](#) 05:19-09:50 - medallion rationale through replayability/idempotency

[OFFICIAL/CURRENT: Medallion architecture in OneLake](#)

[OFFICIAL/CURRENT: Lakehouse end-to-end tutorial](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

For parcel scans, Bronze keeps source JSON/CSV or raw Delta with ingestion metadata. Silver enforces `parcel_id/event_time/event_type`, normalizes timestamps to UTC, deduplicates a defined event key and quarantines malformed records. Gold creates `current_parcel_status` and `daily_hub_volume` with documented grains.

The practice script reads a small contract registry and validates that Gold does not directly depend on the raw source. This is architectural lint, not a substitute for data tests, but it makes layer violations visible.

What you should see

You can define one-sentence grain, quality and allowed-consumer rules for each layer and explain how to replay Silver/Gold from preserved evidence.

Why it worked

The layers represent increasing trust and different contracts. That makes incident recovery and ownership more precise than a single folder called `cleaned_data_final_v7`.

Break it / common failure

Anti-patterns include transforming Bronze until it is no longer raw, letting reports query unstable Bronze directly, copying the same cleansed table into multiple Gold folders, and treating Gold as 'aggregated only.' A Gold table can be detailed if it is consumer-ready and contractually stable.

Reason through a realistic incident

A new source field breaks Silver parsing. Bronze still contains the original payloads. Pause promotion to Gold, update the Silver contract deliberately, replay the affected Bronze interval, reconcile counts and then resume. If Bronze had been overwritten, you would be dependent on the source resending history.

Deepen the engineering reasoning

Fabric gives several deployment patterns: separate lakehouses/workspaces per layer or mixed lakehouse/warehouse patterns. Choose the separation that makes security, ownership and lifecycle clear for your team. Layer boundaries should also carry observability: input count, valid count, quarantine count, dedupe count and output count. A useful identity is `input = valid_written + quarantined + intentionally_filtered` with each category explainable. Medallion is not a license to build three copies of every table forever; use it where progressive quality and replay boundaries create real value.

Try it yourself

Define Bronze/Silver/Gold contracts for ecommerce orders. Include grain, required keys, one quality check and allowed consumers for each layer.

Fresh changed retry

A regulatory consumer requires immutable monthly snapshots. Add that requirement without turning every Gold table into a frozen copy.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Medallion = progressive trust and replay boundaries, not three fashionable folder names. Preserve raw evidence; make Silver quality explicit; make Gold consumer contracts stable.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-12

Incremental loads and MERGE in cloud

Watch first

[1. Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#)

01:22:35-01:32:40 - Lookup activity; 02:33:15-02:43:55 - stored procedures in pipelines

[2. Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#)

02:31:55-02:45:14 - metadata-driven notebooks before schema-drift lesson

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 12 of 16 | Primary practice:

practice/fabric_patterns/12_incremental_merge_simulator.py

Why this matters

Cloud pipelines become expensive and fragile if every run rereads all history. Incremental loading reduces work, but it introduces state: watermarks, change identifiers, late updates and replay boundaries. M08/M13 taught the logic; M15 combines it with Fabric pipelines and Delta MERGE patterns.

Picture it in your head

Think of a bookmark plus a safety margin. The bookmark says how far the pipeline has durably processed. The margin or change feed protects against records that arrive late or are corrected near the boundary. MERGE then applies identified changes using a stable business key.

Words before we use them

Word	Meaning before we use it
watermark	Durable progress value such as last processed timestamp/ID.
candidate watermark	New high-water mark observed for the current run but not yet committed.
lookback	Re-read interval used to capture late changes near the boundary.
upsert	Insert new keys and update existing keys.
MERGE	Delta/SQL pattern matching source changes to target keys for conditional insert/update/delete.
idempotency	Same logical input can be replayed without creating an incorrect extra effect.

Useful video / source route

[VIDEO: Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#)

01:22:35-01:32:40 - Lookup activity; 02:33:15-02:43:55 - stored procedures in pipelines

[VIDEO: Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#)

02:31:55-02:45:14 - metadata-driven notebooks before schema-drift lesson

[OFFICIAL/CURRENT: Incremental copy tutorial](#)

[OFFICIAL/CURRENT: Fabric Copy activity](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

A source orders table has LastModifyTime. The pipeline reads committed watermark 2026-09-13T00:00Z and current source maximum 2026-09-14T00:00Z. It copies rows (old, candidate] to a durable staging area, validates them, then a notebook MERGES by order_id into the silver Delta table. Only after target reconciliation passes does the pipeline commit the candidate watermark.

The simulator also introduces a tied timestamp and replay. With a stable key/upsert, replaying boundary rows is acceptable; using > carelessly can skip a record that appears later with the prior max timestamp.

```
MERGE INTO silver_orders AS t
USING staged_changes AS s
ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

What you should see

You can draw candidate versus committed progress, name the idempotency key, and explain what happens on failure before and after each durable step.

Why it worked

The checkpoint belongs to validated durable output, not to source observation. MERGE makes re-read rows safe when the match key and update semantics are correct.

Break it / common failure

Never set the watermark to source max before copied rows are durably saved and validated. Avoid assuming timestamps are unique/monotonic. A lookback window reduces some late-data risk but does not solve arbitrarily late corrections unless the source exposes a reliable change sequence/CDC contract.

Reason through a realistic incident

A job failed after staging but before MERGE. The candidate watermark exists in logs but committed progress is unchanged. Safe recovery is to replay the interval from the old committed watermark, reconcile staging and apply idempotent MERGE. Do not manually commit the candidate because 'most rows probably made it.'

Deepen the engineering reasoning

Incremental design has three separate contracts: source-change detection, durable landing, and target application. Each may fail independently. Keep run IDs and source boundaries so you can prove which changes were observed. For large pipelines, partition-aware replay or source CDC can be better than timestamp lookback. The target must also define delete semantics - soft delete, tombstone or physical delete - rather than treating MERGE as automatically correct. Finally, reconcile not just row count but business keys and change categories.

Try it yourself

Design incremental loading for customers with `updated_at`, `customer_id` and `is_deleted`. State selection predicate, MERGE key, delete handling and commit point.

Fresh changed retry

Now assume `updated_at` can be corrected backwards by the source. Explain why the original watermark alone is insufficient and propose a safer source/change contract.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Incremental correctness = reliable change boundary + durable landing + idempotent target apply + commit progress only after validation.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

Compare versions before committing progress

An initial file is a source batch, not permission to overwrite the existing table. The guided notebook keeps one current order per order_id, accepts strictly newer versions and rejects equal-version disagreements. It checks the projected row count and paid total before writing. A later crash can still leave newer Silver and older Gold because those are separate transactions; replay repairs that state. The committed timestamp advances only after both outputs pass validation.

DE15-13

Permissions, connections and secrets

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

01:22:30-01:50:53 - security and governance

[2. Aleksi Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#)

25:03-41:38 - NotebookUtils and platform utilities

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 13 of 16 | Primary practice: `practice/fabric_patterns/13_security_secret_patterns.py`

Why this matters

Cloud pipelines need credentials to reach external systems, but credentials are not normal configuration. A password copied into notebook source, pipeline JSON or Git history becomes a long-lived incident. Fabric supports managed connections and NotebookUtils credential helpers; Azure Key Vault can hold secrets where required. The design goal is to separate code from secret material and make runtime identity auditable.

Picture it in your head

Code should know the name of a locked cabinet and the label of a key - not contain the key itself. The runtime identity opens the cabinet if authorized. Logs should show which identity and connection were used without printing secret values.

Words before we use them

Word	Meaning before we use it
connection	Fabric-managed configuration for reaching a data source/destination.
credential	Secret or identity proof used by a connection.
secret	Sensitive value such as password/token/key.
Key Vault	Azure service for securely storing secrets/keys/certificates.

managed identity/service principal

Non-human workload identity used for automation.

secret rotation Replacing a credential without rewriting business logic.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 01:22:30-01:50:53 - security and governance

[VIDEO: Aleksii Partanen - Learn Microsoft Fabric Notebooks in 2026 - Full Course!](#) 25:03-41:38 - NotebookUtils and platform utilities

[OFFICIAL/CURRENT: NotebookUtils credentials](#)

[OFFICIAL/CURRENT: Fabric Spark security best practices](#)

[OFFICIAL/CURRENT: Fabric workspace roles](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

The bad pattern is `PASSWORD= 'P@ssw0rd! '` inside a notebook. The safer pattern is a managed Fabric connection or a Key Vault secret retrieved at runtime by an authorized identity. The notebook references a secret name, not the secret value. Logs record `connection=crm_prod, identity=svc-fabric-prod, result=success` without printing tokens.

The practice linter scans example configuration strings for suspicious hard-coded secret patterns and demonstrates a redacted configuration. It cannot prove cloud security, but it trains the habit that source files should remain safe to commit.

```
# Pattern only - never print the returned secret
secret = notebookutils.credentials.getSecret(vault_uri, secret_name)
# Prefer managed/federated identity or managed connections when possible.
```

What you should see

You can point to where identity is established, where secret material is stored, how it is retrieved, and what permissions allow that retrieval.

Why it worked

Secret rotation becomes independent of code deployment. A compromised code repository does not automatically reveal production credentials when secrets are externalized correctly.

Break it / common failure

Never print secrets to confirm they are correct. Avoid storing them in notebook output, `.env` committed to Git, pipeline parameter defaults, screenshots or README files. Also understand that having permission to manage a Key Vault resource is not necessarily the same as permission to read secret values.

Reason through a realistic incident

A notebook fails with 403 when reading Key Vault after promotion. DEV worked under your personal identity. Compare PROD execution identity and Key Vault data-plane role assignment. Do not paste the secret directly into the notebook as a workaround.

Deepen the engineering reasoning

Production security is layered: workspace role, item permission, connection permission, external source permission and network path can all matter. Least privilege should cover both control plane and data plane. Use service identities for scheduled production jobs and design a rotation procedure before the first emergency. Keep secrets out of diagnostic bundles; if a library redacts notebook outputs, still avoid relying solely on redaction. The safest secret is one your application never receives because a managed identity can authenticate directly.

Try it yourself

Review five configuration values - workspace name, table name, API base URL, API token, Key Vault secret name. Classify which may live in source control and which must not.

Fresh changed retry

Replace a client-secret-based service principal with a managed/federated identity where the target supports it. Explain what secret disappears and what authorization still remains necessary.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Code references identities/connections/secret names; secret values live outside source. Diagnose 401/403 by identity + permission + scope + network, not by leaking credentials.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-14

Monitoring, reliability and cost awareness

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

04:46:53-05:43:50 - monitoring/optimization of processing tools and data stores

[2. Aleksi Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#)

01:40:09-01:48:17 - scheduling and monitoring pipelines

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 14 of 16 | Primary practice: `practice/fabric_patterns/14_monitoring_cost_simulator.py`

Why this matters

A cloud pipeline that runs successfully but slowly, expensively or unreliably is not production-ready. Fabric exposes run history, Spark monitoring and capacity metrics. Your job is to convert those signals into service-level evidence: freshness, duration, failure rate, backlog, resource pressure and cost/Capacity Unit consumption.

Picture it in your head

Think of three dashboards. The run dashboard answers 'did this job finish and what failed?' The Spark detail view answers 'what did distributed compute spend time doing?' The capacity metrics app answers 'is shared compute overloaded and which operations consume it?' You need all three levels for serious diagnosis.

Words before we use them

Word	Meaning before we use it
SLA/SLO	Target for service availability, freshness or performance.
freshness	How current the delivered data is relative to source/business time.
CU	Capacity Unit, Fabric measure of compute capacity/consumption.
throttling	Service slowing/restricting work when capacity limits are exceeded.
run history	Recorded executions with status, duration and diagnostics.
baseline	Normal expected behavior used to identify regression.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 04:46:53-05:43:50 - monitoring/optimization of processing tools and data stores

[VIDEO: Aleksii Partanen - Learn Microsoft Fabric Data Pipelines in 2026 - Full Course!](#) 01:40:09-01:48:17 - scheduling and monitoring pipelines

[OFFICIAL/CURRENT: Spark monitoring overview](#)

[OFFICIAL/CURRENT: Fabric Capacity Metrics app](#)

[OFFICIAL/CURRENT: Pipeline pricing](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Suppose the nightly silver build normally finishes in 18 minutes and consumes a stable capacity footprint. It now takes 55 minutes. Start with run evidence: did source volume triple, did one activity retry, or did Spark time expand? Then inspect Spark jobs for skew/shuffle from M12. Finally inspect capacity metrics for overload/throttling and competing workloads.

The local simulator accepts `rows_processed`, `duration_seconds` and `retry_count` and calculates throughput plus a simple alert state. It intentionally does not claim to calculate Fabric billing; actual cost/consumption comes from Fabric metrics/pricing evidence.

What you should see

You can distinguish job-level failure, Spark performance regression and shared-capacity pressure, and you know which Fabric monitoring surface to consult for each.

Why it worked

Diagnosis follows layers and baselines. You compare current evidence against known normal behavior rather than making a blind optimization change.

Break it / common failure

Do not respond to every slowdown by scaling capacity. A Cartesian join or tiny-file explosion can waste any capacity. Conversely, do not micro-optimize code while the capacity is visibly throttled by unrelated workloads. Also do not use a single successful run as a reliability claim.

Reason through a realistic incident

Gold is two hours stale but every pipeline run is green. Investigate scheduling gaps, source watermark movement and whether the pipeline processed zero rows due to a logic bug. Reliability requires semantic freshness checks, not just orchestration status.

Deepen the engineering reasoning

Define operational metrics before incidents: `last_success_time`, `max_source_event_time`, `target_max_event_time`, input/output/quarantine counts, duration, retry count and run ID. For cost awareness, focus on avoidable work: full scans where incremental works, excessive notebook startups, unbounded debug displays, inefficient shuffles, unnecessary copies and over-frequent schedules. Capacity metrics can show where CU consumption concentrates. Optimization should preserve correctness; a cheaper pipeline that skips late data is not an optimization.

Try it yourself

Create an M15 dashboard spec with six metrics and alert thresholds for an hourly ingestion/transformation pipeline.

Fresh changed retry

The job duration doubles but source volume is unchanged and capacity is healthy. List the next three pieces of Spark/data-shape evidence you would inspect before touching capacity size.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Observe run correctness, Spark behavior and capacity pressure separately. Optimize from evidence and keep freshness/reconciliation beside performance/cost.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-15

Git, deployment and environment separation

Watch first

[1. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

00:48:26-01:22:30 - CI/CD and lifecycle management

[2. TECHIES STUDY - Fabric Git integration Process and Deployment pipelines](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 15 of 16 | Primary practice:
practice/fabric_patterns/15_deployment_binding_simulator.py

Why this matters

A production data platform should not depend on manually recreating DEV changes in PROD. Fabric supports Git integration and deployment pipelines for many data-engineering items. The difficult part is not clicking Deploy; it is deciding what is versioned, what is data, what must be rebound across environments and what must never be promoted unchanged.

Picture it in your head

Git stores the blueprint; deployment pipelines move supported blueprints between rooms. The furniture/data already in the target room is not automatically replaced. Environment-specific bindings - lakehouse IDs, connections, credentials, capacities - may need logical binding or deployment rules.

Words before we use them

Word	Meaning before we use it
Git integration	Sync between supported Fabric item definitions and a Git repository.
deployment pipeline	Fabric lifecycle tool for promoting supported content across stages.
logical ID/binding	Reference used to map a dependency to the corresponding target item.
deployment rule	Target-stage override for supported configuration/binding.
DEV/TEST/PROD	Separated environments used to reduce production risk.
metadata deployment	Promotion of item definitions/configuration rather than business data copies.

Useful video / source route

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) 00:48:26-01:22:30 - CI/CD and lifecycle management

[VIDEO: TECHIES STUDY - Fabric Git integration Process and Deployment pipelines](#) Optional visual - no verified bounded range

[OFFICIAL/CURRENT: Fabric Git integration overview](#)

[OFFICIAL/CURRENT: Lakehouse Git/deployment behavior](#)

[OFFICIAL/CURRENT: Notebook source control and deployment](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Connect a DEV workspace to Git. Version the lakehouse metadata, notebook source and environment configuration that Fabric supports. TEST/PROD are separate workspaces. When promoting the notebook, ensure its default lakehouse/environment bind to the target equivalents rather than pointing back to DEV. Business table data is not 'deployed from Git' just because lakehouse metadata is versioned.

The simulator holds a mapping of logical names to DEV and PROD physical IDs. A validation function fails if the deployed notebook in PROD still references a DEV physical ID. This models one of the most important cloud-release checks without needing to fake a Fabric deployment locally.

What you should see

You can explain what Git tracks, what deployment moves, what remains environment-specific and how you would verify that PROD dependencies point to PROD resources.

Why it worked

Definitions and data are separated. Promotion becomes repeatable while persistent production data stays under the target environment's lifecycle.

Break it / common failure

Do not commit notebook outputs/secrets. Do not assume every Fabric item supports identical Git/deployment behavior. Do not treat a successful metadata deployment as proof that connections and downstream data contracts are correct. And never let PROD notebooks silently reference DEV lakehouses because an ID was hard-coded.

Reason through a realistic incident

A PROD deployment succeeds, but the notebook reads DEV data. Stop the run, preserve evidence, inspect default-lakehouse binding and deployment rules, and correct the binding. This is a release-validation failure, not a reason to copy DEV data into PROD.

Deepen the engineering reasoning

Good deployment separates three classes of state: source-controlled definitions, environment configuration/bindings, and durable business data. Review each dependency at release time. Use pull requests and code review for notebook/pipeline changes where possible. Maintain small smoke tests that run after promotion with target-safe fixtures. Because Fabric's supported-item list evolves, treat current Microsoft documentation as authority rather than assuming an old blog's CI/CD matrix is permanent.

Try it yourself

List which parts of a lakehouse+notebook+pipeline solution you expect in Git and which should be environment-specific. Add one post-deploy smoke test.

Fresh changed retry

The target workspace uses a different lakehouse name and connection. Design a logical mapping/deployment-rule approach instead of editing notebook source after every deployment.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

Git versions supported definitions; deployment promotes metadata; bindings/config map dependencies per environment; production data is not a source-control artifact.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

DE15-16

End-to-end Fabric data engineering case

Watch first

[1. Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)

00:00-18:30 - end-to-end lakehouse + pipeline + analysis walkthrough

[2. Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#)

Optional visual - no verified bounded range

[3. Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#)

Optional visual - no verified bounded range for the full course; use selected chapters from prior lessons

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M15 lesson 16 of 16 | Primary practice: `practice/fabric_patterns/16_end_to_end_local_case.py`

Open full guided Fabric build

Why this matters

The final lesson is where isolated knowledge becomes engineering. You will connect identity, workspace design, OneLake, lakehouse, notebooks/Spark, pipelines, incremental state, SQL serving, medallion, monitoring and deployment into one coherent case. A portfolio project becomes credible when every component has a reason and every state transition has evidence.

Picture it in your head

Trace one order change from source to consumer. Ask six questions at every boundary: who authenticates, where is it durable, what transforms it, how is progress recorded, how is correctness validated, and how would you replay after failure? If you can answer those, the architecture is understandable.

Words before we use them

Word	Meaning before we use it
end-to-end lineage	Trace of data and control dependencies from source to consumer.
run contract	Inputs, outputs, progress state and success criteria for one execution.
promotion evidence	Proof that a release points to the correct target environment and passes smoke tests.
reconciliation	Comparison that proves output population/keys/measures agree with expected source/change boundaries.
operational runbook	Instructions for normal run, failure diagnosis and safe recovery.
portfolio evidence	Artifacts/screenshots/code/results that demonstrate work rather than only describing it.

Useful video / source route

[VIDEO: Amit Chandak - Microsoft Fabric: Load data in Lakehouse using Data Pipeline | End to End](#)
00:00-18:30 - end-to-end lakehouse + pipeline + analysis walkthrough

[VIDEO: Vijay Perepa - Microsoft Fabric Data Engineering Explained: Lakehouse, Spark and OneLake Architecture](#) Optional visual - no verified bounded range

[VIDEO: Learn Microsoft Fabric with Will - DP-700 Exam Full Course](#) Optional visual - no verified bounded range for the full course; use selected chapters from prior lessons

[OFFICIAL/CURRENT: Fabric lakehouse end-to-end scenario](#)

[OFFICIAL/CURRENT: Fabric Data Engineering overview](#)

Use rule: Videos are strongly encouraged because this course is designed for visual learning. Current official Microsoft docs are the authority for portal/runtime/security behavior. If a video is older, use it for the mental model/UI flow and then reconcile commands/capabilities with the current docs.

Follow with me once

Case: a regional ecommerce company has an operational orders database and daily product files. DEV Fabric workspace contains bronze_lh, silver_gold_lh, ingest_orders pipeline, transform_orders notebook, validation notebook and a Fabric Environment. The pipeline reads the old/new watermark, copies changes to Bronze with run_id metadata, validates landing, calls the transform notebook to MERGE Silver, rebuilds affected Gold daily_sales partitions, reconciles counts/amounts, then commits progress. Analysts query Gold through the SQL analytics endpoint. A shortcut exposes a centrally owned product dimension without copying it. Secrets remain in managed connections/Key Vault.

Monitoring records last_success, source watermark, committed watermark, row counts, quarantine count, duration and capacity evidence. Git tracks supported definitions; deployment promotes to TEST/PROD with target bindings. A runbook states what to do if copy succeeds but transform fails: leave committed watermark

unchanged and replay from the old boundary.

Run `16_end_to_end_local_case.py`. It uses local CSV/JSON state to simulate two runs, including a replay. The expected proof is no duplicate order keys, committed watermark advances only after validation, and Gold total reconciles with Silver valid orders. This does not pretend to be Fabric execution; it proves the control logic before you reproduce it in Fabric.

What you should see

You can narrate the architecture from one source change to one consumer query, including identity, durable state, validation, failure recovery, monitoring and deployment.

Why it worked

Each component owns a clear responsibility and durable boundary. The project can be debugged because progress and reconciliation evidence survive failures.

Break it / common failure

A flashy portal screenshot is not portfolio proof if you cannot explain why each item exists. Avoid adding Eventstream, Warehouse, Dataflow, Databricks or AI components merely to increase tool count. Complexity must earn its place through a requirement.

Reason through a realistic incident

Production Gold is stale. The ingest pipeline says success, Silver is current, and Gold is not. The architecture narrows the incident to the Silver->Gold step or its scheduling/validation path. Because Bronze/Silver evidence and committed watermark are intact, you can repair Gold without re-pulling the source or guessing at missing data.

Deepen the engineering reasoning

Your portfolio write-up should include architecture diagram, source contract, incremental-state design, one deliberate failure/recovery, reconciliation evidence, security approach, runtime evidence, monitoring screenshot and deployment explanation. Include limitations honestly: if you used a Fabric trial, say so; if managed private networking was not available, describe rather than fake it. Add one cost-conscious decision, such as incremental copy instead of full reload. Add one security-conscious decision, such as workload identity/managed connection instead of hard-coded secret. That is stronger than claiming 'enterprise grade' without evidence.

Try it yourself

Write a one-page design for the same case before looking back at the worked solution. Include items, layers, identities, progress state, serving surface and monitoring evidence.

Fresh changed retry

Change the requirement: product data must remain in an external ADLS Gen2 account and finance requires a writable SQL gold mart. Update the design using a OneLake shortcut and Fabric Warehouse only where justified.

Validation and recovery ladder

- Confirm the active identity, workspace/item and environment before changing code.
- Name the durable input, durable output and progress/checkpoint state for this lesson.
- Reconcile counts/keys/measures or permission evidence; a green portal status alone is not proof.
- If the operation may be replayed, prove the write/side effect is safe to retry before calling recovery complete.
- Keep the genuine first failure. A repaired run is repair evidence; the changed retry is mastery evidence.

Revision in your own words

End-to-end DE is a chain of explicit contracts. The strongest project proves state, replay, reconciliation, security, observability and release boundaries - not just screenshots.

Evidence to keep

Save: one architecture/path/config/code artifact, one expected output or portal evidence, one failure/recovery note, and a short explanation of the guarantee in your own words. For live Fabric tasks, record workspace/item/runtime/run ID where applicable. You are ready only when you can solve the fresh retry with the review closed.

M15 real Fabric guided build - supplied orders data

This guided walkthrough is for DE15-04, DE15-09 and DE15-16. It connects the course's own files to a real Fabric lakehouse, notebook, pipeline and SQL result.

Target proof: after initial + changes runs, `silver_orders` has **5 unique orders** and the paid revenue is **460.00**. Do not mark LIVE FABRIC PASS unless you actually obtain the portal/run/table evidence in your own tenant.

0. Access and cost guardrail

Preferred learning route: use an eligible Microsoft Fabric trial or an existing Fabric-enabled workspace. Microsoft currently documents a 60-day trial, but eligibility depends on tenant/region/settings and can change.

If you cannot start a trial, **do not buy capacity just to satisfy this course**. Complete the local `fabric_patterns/16_end_to_end_local_case.py` control, study this walkthrough, and record LIVE FABRIC = PENDING - ACCESS UNAVAILABLE. Return to the real build when you have legitimate trial/workspace access.

Official references:

- Trial: <https://learn.microsoft.com/en-us/fabric/get-started/fabric-trial>

- Lakehouse tutorial:

<https://learn.microsoft.com/en-us/fabric/data-engineering/tutorial-lakehouse-get-started/>

- OneLake/lakehouse upload:

<https://learn.microsoft.com/en-us/fabric/onelake/create-lakehouse-onelake>

- Notebook activity: <https://learn.microsoft.com/en-ca/fabric/data-factory/notebook-activity>

- Pipeline parameters: <https://learn.microsoft.com/en-us/fabric/data-factory/parameters>

1. Create the DEV workspace and lakehouse

1. Sign in to Fabric and switch to the Fabric experience.
2. Create a workspace named de2026-m15-dev (add your initials if the name must be unique).
3. Assign the workspace to your eligible trial/paid Fabric capacity.
4. In the workspace select **New item -> Lakehouse**.
5. Name it orders_lh.
6. Open the lakehouse. Confirm you can see both **Files** and **Tables**.

Evidence: save the workspace name and lakehouse name. Do not screenshot secrets or tenant IDs unnecessarily.

2. Upload the supplied course files into OneLake

From the extracted course package use:

- 14_M15/practice/data/orders_initial.csv

- 14_M15/practice/data/orders_changes.csv

In the lakehouse Files area create m15/orders/ and upload both files so the notebook paths are:

Files/m15/orders/orders_initial.csv

Files/m15/orders/orders_changes.csv

Do not rename them. Open/preview each file and confirm the headers include: order_id, updated_at, customer_id, amount, status.

3. Create notebook nb_m15_orders

Create a notebook in the same workspace, attach orders_lh as the default lakehouse, and add the following cells in order.

Use the complete companion file FABRIC_GUIDED_BUILD_NOTEBOOK.py. It is divided into five cells. Copy each cell below in order. Mark only Cell 1 as the parameter cell.

Cell 1 - PARAMETERS: mark THIS cell as the Fabric parameter cell.

```
p_batch = "initial"
p_expected_rows = 3
p_expected_paid_revenue = "200.50"
p_fail_after_silver = False
```

Cell 2 - READ AND VALIDATE; no table changes here.

```

from decimal import Decimal
from pyspark.sql import functions as F, types as T, Window
spark.conf.set("spark.sql.session.timeZone", "UTC")
if p_batch not in {"initial", "changes", "conflict", "newer", "interrupt"}:
    raise ValueError("unsupported batch; use one supplied course file")
fields = ["order_id", "updated_at", "customer_id", "amount", "status"]
schema = T.StructType([T.StructField(x, T.StringType()) for x in fields])
raw = (spark.read.option("header", True).option("enforceSchema", False)
      .schema(schema).csv(f"Files/m15/orders/orders_{p_batch}.csv"))
if raw.limit(1).count() == 0:
    raise ValueError("empty batch: inspect the file before advancing progress")
staged = (raw.withColumn("event_ts", F.try_to_timestamp("updated_at"))
        .withColumn("amount_dec", F.expr("try_cast(amount AS DECIMAL(12,2))")))
invalid = F.lit(False)
for name in fields:
    invalid = invalid | F.col(name).isNull() | (F.length(F.trim(F.col(name))) == 0)
invalid = (invalid | F.col("event_ts").isNull()
        | ~F.col("updated_at").rlike(r"^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(\.\d{1,6})?Z$")
        | F.col("amount_dec").isNull()
        | ~F.col("amount").rlike(r"^\d{1,10}(\.\d{1,2})?$")
        | ~F.col("status").isin("NEW", "PAID", "CANCELLED"))
bad = staged.filter(invalid)
if bad.limit(1).count():
    bad.show(truncate=False)
    raise ValueError("invalid input: no business table or checkpoint was changed")
incoming_all = staged.select("order_id", "customer_id",
                            F.col("amount_dec").alias("amount"), "status", "event_ts").dropDuplicates()
# Same key/version with different content is ambiguous, even if order differs.
conflicts = incoming_all.groupBy("order_id", "event_ts").count().filter("count > 1")
if conflicts.limit(1).count():
    conflicts.show(truncate=False)
    raise ValueError("equal-version conflict inside the input")
w = Window.partitionBy("order_id").orderBy(F.col("event_ts").desc())
incoming = incoming_all.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")

```

Cell 3 - COMPARE WITH EXISTING STATE; initial is NOT an overwrite command.

```

exists = spark.catalog.tableExists("silver_orders")
if not exists and p_batch != "initial":
    raise ValueError("bootstrap once with initial before applying changes")
current = spark.table("silver_orders") if exists else incoming.limit(0)
if current.groupBy("order_id").count().filter("count > 1").limit(1).count():
    raise ValueError("target key conflict: inspect Silver before writing")
s, t = incoming.alias("s"), current.alias("t")
paired = s.join(t, F.col("s.order_id") == F.col("t.order_id"), "inner")
changed = F.lit(False)
for name in ("customer_id", "amount", "status"):
    changed = changed | ~F.col("s." + name).eqNullSafe(F.col("t." + name))
if paired.filter((F.col("s.event_ts") == F.col("t.event_ts")) & changed).limit(1).count():
    raise ValueError("equal-version conflict against Silver; no write allowed")
updates = (s.join(t, F.col("s.order_id") == F.col("t.order_id"), "left")
        .filter(F.col("t.order_id").isNull() | (F.col("s.event_ts") > F.col("t.event_ts"))))
        .select("s.*")
projected = current.join(updates.select("order_id", "order_id", "left_anti").unionByName(updates)

```

Cell 4 - CHECK THE PROPOSED RESULT BEFORE WRITING.

```

row_count = projected.count()
paid = projected.filter("status = 'PAID']").agg(F.sum("amount")).first()[0]
paid = Decimal(str(paid or 0)).quantize(Decimal("0.01"))
expected = Decimal(str(p_expected_paid_revenue))
if not expected.is_finite() or row_count != int(p_expected_rows) or paid != expected:
    raise ValueError(f"reconciliation failed BEFORE write: rows={row_count}, paid={paid}")
# A checkpoint is a high-water mark over processed versions, not a wall-clock time.
row_max = incoming.agg(F.max("event_ts")).first()[0]
control_exists = spark.catalog.tableExists("m15_pipeline_control")
if control_exists:
    ctl = spark.table("m15_pipeline_control")
    if set(ctl.columns) != {"pipeline", "committed_ts"}:
        raise ValueError("old control schema: use a fresh disposable M15 lakehouse")
    previous = ctl.filter("pipeline = 'orders']").agg(F.max("committed_ts")).first()[0]
else:
    previous = None
candidate = max(x for x in (previous, row_max) if x is not None)
print({"before": str(previous), "candidate": str(candidate), "rows": row_count, "paid": str(paid)})

```

Cell 5 - DURABLE SILVER, GOLD, THEN PROGRESS. Single-writer lab.

```

incoming.createOrReplaceTempView("m15_incoming")
if not exists:
    # errorifexists prevents a concurrent/bootstrap rerun from replacing a table.
    incoming.write.format("delta").mode("errorifexists").saveAsTable("silver_orders")
else:
    spark.sql("""
        MERGE INTO silver_orders t USING m15_incoming s
        ON t.order_id = s.order_id
        WHEN MATCHED AND s.event_ts > t.event_ts THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)
if str(p_fail_after_silver).lower() == "true":
    raise RuntimeError("DELIBERATE_FAILURE_AFTER_SILVER: Gold and progress remain unchanged")
# Reconcile actual durable Silver; another writer is not permitted in this lab.
silver = spark.table("silver_orders")
actual_paid = silver.filter("status = 'PAID']").agg(F.sum("amount")).first()[0] or Decimal("0")
if silver.count() != row_count or actual_paid != paid:
    raise ValueError("durable Silver differs: stop before Gold/progress")
gold = spark.createDataFrame([(row_count, paid)], "unique_orders long, paid_revenue decimal(18,2)")
gold.write.format("delta").mode("overwrite").saveAsTable("gold_order_summary")
new_control = spark.createDataFrame([("orders", candidate)], "pipeline string, committed_ts timestamp")
if not control_exists:
    new_control.write.format("delta").mode("errorifexists").saveAsTable("m15_pipeline_control")
else:
    new_control.createOrReplaceTempView("m15_next_control")
    spark.sql("""
        MERGE INTO m15_pipeline_control t USING m15_next_control s
        ON t.pipeline = s.pipeline
        WHEN MATCHED AND s.committed_ts > t.committed_ts THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)
print({"rows": row_count, "paid_revenue": str(paid), "committed": str(candidate)})

```

The proposed output is checked before the first write. Equal-version disagreements fail explicitly. An older initial file cannot overwrite newer orders. Silver, Gold and the checkpoint are separate Delta transactions: keep this teaching pipeline single-writer, and let consumers use it only after the whole run succeeds. A failed run can leave newer Silver with older Gold; replay repairs that boundary.

4. Run interactively once

Run the notebook with defaults (initial). Confirm:

- silver_orders = 3 keys;
- Gold = 3 unique orders, 200.50 paid revenue;
- control table has the initial maximum committed_ts.

Then manually change the parameter values to changes, 5, 460.00 and run again. Confirm the target result **5 / 460.00**.

This interactive run proves the notebook before orchestration.

5. Build pipeline pl_m15_orders

1. Bootstrap interactively once, as in section 4. Confirm the initial run is 3 / 200.50 and the changes run is 5 / 460.00.
2. In the same workspace choose **New item -> Data pipeline** and name it pl_m15_orders.
3. Add one **Notebook** activity named Apply changes, choosing nb_m15_orders.
4. Set Base parameters: p_batch = changes, p_expected_rows = 5, p_expected_paid_revenue = 460.00, and p_fail_after_silver = False.
5. Save and run the pipeline. It replays the changes safely over the existing table.
6. Open the activity output and compare its counts/revenue/checkpoint with the interactive result.

Do not schedule an initial-load overwrite before every incremental run. Initial is a bootstrap input, not a reset operation. If you later replay the initial file over the final state, use expected controls 5 / 460.00: those are the controls of the resulting target, not the three-row input file. For a genuinely fresh demonstration, create a separate disposable lakehouse rather than deleting trusted output.

6. Query the result from the SQL analytics endpoint

Open the SQL analytics endpoint created with orders_lh and run:

```
SELECT COUNT(*) AS silver_rows  
FROM dbo.silver_orders;
```

```
SELECT unique_orders, paid_revenue  
FROM dbo.gold_order_summary;
```

Expected final result:

```
silver_rows = 5  
unique_orders = 5  
paid_revenue = 460.00
```

If the table is not immediately visible, refresh the explorer/endpoint metadata and confirm the notebook wrote managed Delta tables to the attached lakehouse.

7. Deliberate failure + recovery

You need a real failure, not only a green screenshot.

1. In Apply changes, temporarily set `p_expected_paid_revenue` to 999.00.
2. Run the pipeline. The changes activity should fail at reconciliation.
3. Inspect the activity output/notebook run and save the failing expected-vs-observed evidence.
4. Confirm the control table **did not get a new `committed\_ts` from the failed validation**.
5. Restore the expected revenue to 460.00.
6. Rerun. The MERGE is safe to replay and final Gold must be 5 / 460.00.

The deliberately wrong 999.00 control now fails before any write. To test a different boundary, set `p_fail_after_silver` to True: the activity fails after Silver, while Gold and committed progress remain unchanged. Restore False and rerun the same batch; it must converge. For a visibly newer checkpoint, upload `orders_newer.csv` and use expected controls 5 / 525.00 before this crash/recovery test. Do not run two copies concurrently.

8. Changed task - do it without copying the walkthrough

Create a second pipeline or notebook parameterization for a fictional returns feed:

- `return_id` is the business key;
- statuses are REQUESTED, APPROVED, REFUNDED;
- a later timestamp may update a return;
- Gold reports total refunded amount.

Write your own five-row fixture, include one stale update and one equal-version conflict, and document the conflict policy before running it.

9. Evidence / mastery checklist

Save:

- workspace + lakehouse + notebook + pipeline item names;
- Files path showing both supplied CSVs;
- notebook parameter cell;
- pipeline activity graph and Base parameters;
- successful pipeline run ID;
- Silver count = 5;
- Gold paid revenue = 460.00;
- control `committed_ts` before/after;
- deliberate 999.00 failure and corrected recovery;
- SQL endpoint result;

- one paragraph on why checkpoint commit is last;
- changed returns task attempt.

A simulator PASS is useful preparation but is **not** Fabric runtime proof. If access is unavailable, label the live items pending.

10. Cleanup / cost hygiene

When finished:

1. Stop any schedule you created.
2. Delete disposable test items/workspaces you no longer need, or end the trial if you explicitly want to stop using it.
3. Never leave copied credentials in notebook cells or Git.
4. Keep your evidence/screenshots and code, not unnecessary running resources.

Controlled equal-version conflict

After successfully applying orders_newer.csv (5 orders / 525.00 paid), upload orders_conflict.csv and select p_batch = conflict with the same expected controls. It has the same O1005 version but a different status, so the run must fail before any write. If you run the conflict file before orders_newer.csv, it is instead a newer cancellation; that is a different experiment. Keep the sequence explicit.